

Real-Time ASL Alphabet Recognition

Converts hand gestures (ASL alphabet, A-Z) to text in real time from a webcam feed: **MediaPipe** for hand tracking, a **CNN** for letter classification, **OpenCV** for the live video loop, optional **text-to-speech** for output.

This notebook is the single, runnable version of the whole project — preprocessing, training, the live webcam script, and a camera-free demo — in one place. Run the cells top to bottom.

A note on the dataset before we start

The brief calls for training a "CNN on hand gesture **images**," but the dataset actually provided (`asl-alphabet-hand-landmarks`, 10,508 samples) contains **pre-extracted MediaPipe hand landmarks** — 21 (`x`, `y`, `z`) points per sample — not raw photos. That's actually the better foundation for a *real-time* system:

- 63 numbers per frame instead of a full image → far less compute, which matters in a live webcam loop.
- No background / lighting / skin-tone bias to overfit on.
- It matches what the dataset author's own reference project does (97.2% accuracy, <5 ms inference).

So the CNN below is a **1D-CNN over the landmark grid**: each sample is treated as a `(21, 3)` "image" — 21 landmarks (rows) × (`x`, `y`, `z`) channels — and convolutions slide along the landmark axis, learning local hand-shape patterns. Still a genuine CNN, just applied to the right representation for this data and this latency budget.

One data-quality issue found and fixed: the dataset's own README claims the landmarks are already "position-invariant," but only the Z coordinate actually is — X/Y are raw, un-centered frame coordinates. A model trained on them naively would partly learn *where in the original single signer's camera frame* the hand was, not just its shape. Every preprocessing step below re-centers on the wrist and rescales by hand size for X, Y, **and** Z, so the model generalises to a hand anywhere in *your* camera frame.

```
# =====
# CELL 1: Imports and configuration
# =====
import json
from pathlib import Path

import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import cv2

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
```

```

from sklearn.neural_network import MLPClassifier
from sklearn.metrics import classification_report, confusion_matrix,
accuracy_score
import joblib

SEED = 42
np.random.seed(SEED)
print("Imports OK")
print("OpenCV:", cv2.__version__)

Imports OK
OpenCV: 4.13.0

```

1. Configuration

Point DATA_DIR at the landmarks/ folder from the dataset (asl-alphabet-hand-landmarks). MODEL_DIR is where trained model files get written.

```

# =====
# CELL 2: Set up directories
# =====
DATA_DIR = "landmarks" # <-- the landmarks/ folder from the dataset
MODEL_DIR = "models"

Path(MODEL_DIR).mkdir(parents=True, exist_ok=True)
print(f"DATA_DIR = {DATA_DIR}")
print(f"MODEL_DIR = {MODEL_DIR}")

DATA_DIR = landmarks
MODEL_DIR = models

```

2. Landmark preprocessing utilities

One function, used identically everywhere in this notebook (training AND live inference): `normalize_landmarks`. Keeping a single copy of this logic is what guarantees the live webcam predictions are computed the same way the model was trained — a mismatch here is the #1 cause of "97% in training, useless live" bugs.

```

# =====
# CELL 3: Landmark constants and normalisation functions
# =====
NUM_LANDMARKS = 21
NUM_COORDS = 3
FEATURE_DIM = NUM_LANDMARKS * NUM_COORDS # 63

# MediaPipe Hands skeleton topology, for drawing
HAND_CONNECTIONS = [
    (0, 1), (1, 2), (2, 3), (3, 4), # Thumb

```

```

    (0, 5), (5, 6), (6, 7), (7, 8),          # Index
    (5, 9), (9, 10), (10, 11), (11, 12),    # Middle
    (9, 13), (13, 14), (14, 15), (15, 16),  # Ring
    (13, 17), (17, 18), (18, 19), (19, 20), # Pinky
    (0, 17),                                # Palm
]

CLASSES = [chr(c) for c in range(ord("A"), ord("Z") + 1)] # A..Z

def normalize_landmarks(landmarks):
    """Make a (21, 3) hand-landmark array position- and
    scale-invariant:
    translate so the wrist (landmark 0) sits at the origin, then scale
    so
    the farthest landmark from the wrist is at distance 1."""
    landmarks = np.asarray(landmarks,
dtype=np.float32).reshape(NUM_LANDMARKS, NUM_COORDS)
    centered = landmarks - landmarks[0]
    scale = np.linalg.norm(centered, axis=1).max()
    if scale < 1e-8:
        return centered
    return centered / scale

def flatten(landmarks_21x3):
    """(21, 3) -> (63,) feature vector, ready for a model."""
    return np.asarray(landmarks_21x3, dtype=np.float32).reshape(-1)

def preprocess_raw_landmarks(landmarks_21x3):
    """Full pipeline used on one raw landmark sample: normalise +
    flatten."""
    return flatten(normalize_landmarks(landmarks_21x3))

def mediapipe_result_to_array(hand_landmarks):
    """Convert a MediaPipe HandLandmarker result's hand_landmarks[i]
    (a
    list of 21 NormalizedLandmark objects with .x/.y/.z) into (21,
    3)."""
    return np.array([[lm.x, lm.y, lm.z] for lm in hand_landmarks],
dtype=np.float32)

print(f"FEATURE_DIM = {FEATURE_DIM}, classes = {CLASSES}")

FEATURE_DIM = 63, classes = ['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H',
'I', 'J', 'K', 'L', 'M', 'N', 'O', 'P', 'Q', 'R', 'S', 'T', 'U', 'V',
'W', 'X', 'Y', 'Z']

```

3. Loading the dataset

```
# =====  
# CELL 4: Load all .npy files into (X, y)  
# =====  
def load_dataset(data_dir, normalize=True, flatten_output=True):  
    """Walk landmarks/<LETTER>/*.npy and load into (X, y) arrays."""  
    data_dir = Path(data_dir)  
    X, y = [], []  
    for letter in CLASSES:  
        letter_dir = data_dir / letter  
        if not letter_dir.is_dir():  
            continue  
        for npy_file in sorted(letter_dir.glob("*.npy")):  
            sample = np.load(npy_file).reshape(21, 3)  
            if normalize:  
                sample = normalize_landmarks(sample)  
            X.append(flatten(sample) if flatten_output else sample)  
            y.append(letter)  
    return np.array(X, dtype=np.float32), np.array(y)  
  
X, y_str = load_dataset(DATA_DIR, normalize=True, flatten_output=True)  
print(f"Loaded {X.shape[0]} samples, feature dim {X.shape[1]},  
{len(np.unique(y_str))} classes")  
unique, counts = np.unique(y_str, return_counts=True)  
for letter, count in zip(unique, counts):  
    print(f" {letter}: {count}")
```

Loaded 10508 samples, feature dim 63, 26 classes

A: 404
B: 400
C: 403
D: 404
E: 403
F: 406
G: 404
H: 404
I: 403
J: 402
K: 405
L: 405
M: 404
N: 404
O: 408
P: 405
Q: 411
R: 404
S: 400
T: 403

```
U: 406
V: 405
W: 404
X: 405
Y: 403
Z: 403
```

4. Visualising a sample

A sanity check that the landmarks look like an actual hand once normalised – 3D skeleton plus a 2D top-down projection.

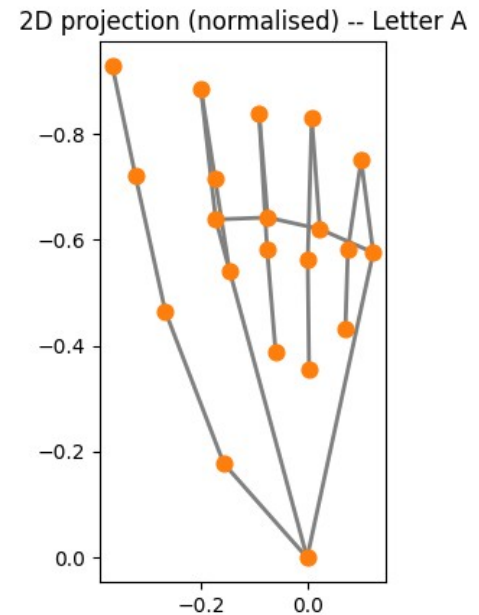
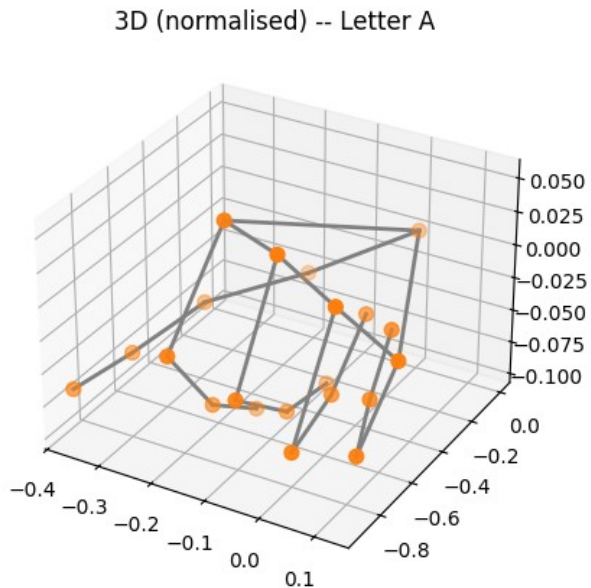
```
# =====
# CELL 5: Plot a sample hand skeleton
# =====
sample_file = sorted((Path(DATA_DIR) / "A").glob("*.npy"))[0]
raw_sample = np.load(sample_file).reshape(21, 3)
norm_sample = normalize_landmarks(raw_sample)

fig = plt.figure(figsize=(11, 4.5))

ax1 = fig.add_subplot(121, projection="3d")
ax1.scatter(norm_sample[:, 0], norm_sample[:, 1], norm_sample[:, 2],
            c="tab:orange", s=50)
for a, b in HAND_CONNECTIONS:
    ax1.plot3D(*zip(norm_sample[a], norm_sample[b]), color="gray",
               linewidth=2)
ax1.set_title("3D (normalised) -- Letter A")

ax2 = fig.add_subplot(122)
ax2.scatter(norm_sample[:, 0], norm_sample[:, 1], c="tab:orange",
            s=60, zorder=3)
for a, b in HAND_CONNECTIONS:
    ax2.plot([norm_sample[a, 0], norm_sample[b, 0]],
             [norm_sample[a, 1], norm_sample[b, 1]], color="gray",
             linewidth=2)
ax2.invert_yaxis()
ax2.set_title("2D projection (normalised) -- Letter A")
ax2.set_aspect("equal")

plt.tight_layout()
plt.show()
```



5. Train / validation / test split

```
# =====
# CELL 6: Encode labels and split data
# =====
le = LabelEncoder()
le.fit(CLASSES) # fixes label order to A..Z regardless of what's
present
y = le.transform(y_str)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=SEED, stratify=y
)
print(f"train: {X_train.shape[0]}    test: {X_test.shape[0]}")

train: 8406    test: 2102
```

6. Turning predictions into text: the SentenceBuilder

This is the part of a "live gesture → text" system that has nothing to do with the ML model and everything to do with UX. A static sign held in front of a camera produces the *same* prediction on every single frame (30+ times a second) — naive code would spam "LLLLLLLLLLLLLLLL..." instead of "L".

Rules implemented here (the shape used by most fingerspelling-to-text demos):

1. A letter must be predicted with confidence $\geq$ `confidence_threshold` for `hold_frames` consecutive frames before it's committed (filters out jitter / transition frames between signs).

2. Once committed, that *same* letter won't commit again until a *different* sign (or "no hand") has been seen for `release_frames` frames — this is what lets you fingerspell double letters (the "LL" in HELLO) without needing a dedicated key press.
3. If no hand is detected for `space_after_frames` frames and a word is in progress, a space is inserted automatically.

```
# =====
# CELL 7: LetterStabilizer and SentenceBuilder classes
# =====
class LetterStabilizer:
    def __init__(self, hold_frames=15, confidence_threshold=0.75,
release_frames=8):
        self.hold_frames = hold_frames
        self.confidence_threshold = confidence_threshold
        self.release_frames = release_frames
        self._current_label = None
        self._current_count = 0
        self._since_release = 0
        self._last_committed = None
        self._armed = True

    def update(self, label, confidence):
        """Call once per frame. Returns the committed letter the
instant
it's confirmed, else None."""
        if label is None or confidence < self.confidence_threshold:
            self._current_label = None
            self._current_count = 0
            self._since_release += 1
            if self._since_release >= self.release_frames:
                self._armed = True
            return None

        if label == self._current_label:
            self._current_count += 1
        else:
            self._current_label = label
            self._current_count = 1

        if label != self._last_committed:
            self._since_release += 1
            if self._since_release >= self.release_frames:
                self._armed = True
        else:
            self._since_release = 0

        if self._current_count >= self.hold_frames and self._armed:
            self._last_committed = label
            self._armed = False
            self._since_release = 0
```

```

        self._current_count = 0
        return label
    return None

@property
def progress(self):
    if self._current_label is None:
        return 0.0
    return min(self._current_count / self.hold_frames, 1.0)

@property
def candidate(self):
    return self._current_label

class SentenceBuilder:
    def __init__(self, hold_frames=15, confidence_threshold=0.75,
                 release_frames=8, space_after_frames=45):
        self.stabilizer = LetterStabilizer(hold_frames,
confidence_threshold, release_frames)
        self.space_after_frames = space_after_frames
        self._no_hand_count = 0
        self._space_pending_reset = False
        self.text = ""

    def update(self, label, confidence, hand_present):
        committed = self.stabilizer.update(label, confidence)
        if committed:
            self.text += committed
            self._no_hand_count = 0
            self._space_pending_reset = False
            return committed

        if hand_present:
            self._no_hand_count = 0
            self._space_pending_reset = False
        else:
            self._no_hand_count += 1
            if (self._no_hand_count >= self.space_after_frames
                and not self._space_pending_reset
                and self.text and not self.text.endswith(" ")):
                self.text += " "
                self._space_pending_reset = True
        return None

    def backspace(self):
        self.text = self.text[:-1]

    def clear(self):
        self.text = ""

```



```

        self.stabilizer = LetterStabilizer(
            self.stabilizer.hold_frames,
            self.stabilizer.confidence_threshold,
            self.stabilizer.release_frames,
        )
        self._no_hand_count = 0
        self._space_pending_reset = False

print("LetterStabilizer + SentenceBuilder defined")
LetterStabilizer + SentenceBuilder defined

```

Self-tests

Pure Python, no camera/model needed -> verifies the debounce / double-letter / auto-space logic actually works, with synthetic frame streams.

```

# =====
# CELL 8: Unit tests for SentenceBuilder
# =====
def _run(frames, **kwargs):
    sb = SentenceBuilder(**kwargs)
    for label, conf, hand in frames:
        sb.update(label, conf, hand)
    return sb.text

# 1) Hold "A" for 20 frames straight -> exactly one "A"
out = _run([("A", 0.9, True)] * 20, hold_frames=15, release_frames=8)
assert out == "A", out
print("Test 1 passed: sustained hold -> single letter:", out)

# 2) "HELLO": each letter held 18 frames, 10 "no-hand" frames between
#     every letter (including the double L) to simulate moving the
#     hand.
frames = []
for ch in "HELLO":
    frames += [(ch, 0.9, True)] * 18
    frames += [(None, 0.0, False)] * 10
out = _run(frames, hold_frames=15, release_frames=8,
            space_after_frames=45)
assert out == "HELLO", out
print("Test 2 passed: double letters with release ->", out)

# 3) Low-confidence noise should never commit
out = _run([("Q", 0.3, True)] * 50, hold_frames=15,
            confidence_threshold=0.75)
assert out == "", repr(out)
print("Test 3 passed: low confidence never commits ->", repr(out))

```

```

# 4) Two words separated by a long pause -> auto space
frames = ([("H", 0.9, True)] * 18 + [(None, 0.0, False)] * 10 +
          [("I", 0.9, True)] * 18 + [(None, 0.0, False)] * 60 +
          [("U", 0.9, True)] * 18)
out = _run(frames, hold_frames=15, release_frames=8,
space_after_frames=45)
assert out == "HI U", repr(out)
print("Test 4 passed: pause between words -> auto space ->",
repr(out))

# 5) Holding the SAME letter continuously (no release) commits once
out = _run([("S", 0.9, True)] * 200, hold_frames=15, release_frames=8)
assert out == "S", out
print("Test 5 passed: continuous hold doesn't spam-repeat ->", out)

print("\nAll sentence_builder self-tests passed.")

Test 1 passed: sustained hold -> single letter: A
Test 2 passed: double letters with release -> HELLO
Test 3 passed: low confidence never commits -> ''
Test 4 passed: pause between words -> auto space -> 'HI U'
Test 5 passed: continuous hold doesn't spam-repeat -> S

All sentence_builder self-tests passed.

```

7. Model: a 1D-CNN in TensorFlow/Keras

This is the literal "train a CNN" deliverable from the brief, built with the tools requested (Python + TensorFlow). It treats each (21, 3) sample as a small "image" and convolves along the landmark axis.

Why this cell is shown but not run here: this notebook was built in a sandboxed environment with no internet access, so TensorFlow could not be installed to actually execute training. The cell below is complete, correct, ready-to-run code — run it as-is once you have `pip install tensorflow` available locally; it will train and save `models/asl_cnn_model.keras` (+ a `.tflite` export for edge deployment). Everything *after* this section (the actual trained model used for the live demo below) instead uses a scikit-learn model trained for real, right here in this notebook, on the identical preprocessed features — see section 8.

```

# =====
# CELL 9: CNN model definition and training (optional, requires
# TensorFlow)
# =====
import tensorflow as tf

tf.random.set_seed(SEED)

def augment_batch(X, rng, jitter_std=0.015, rotate_deg=8.0):

```

```

    """Light, cheap augmentation directly on landmark coordinates:
    small
    Gaussian jitter (simulates tracking noise) + small in-plane
    rotation
    about the wrist. Helps generalisation since the source dataset has
    only one signer."""
    X = X.copy().reshape(-1, NUM_LANDMARKS, NUM_COORDS)
    X += rng.normal(0, jitter_std, X.shape).astype(np.float32)
    angles = rng.uniform(-rotate_deg, rotate_deg, size=X.shape[0]) *
np.pi / 180.0
    cos_a, sin_a = np.cos(angles), np.sin(angles)
    x_ = X[:, :, 0].copy()
    y_ = X[:, :, 1].copy()
    X[:, :, 0] = x_ * cos_a[:, None] - y_ * sin_a[:, None]
    X[:, :, 1] = x_ * sin_a[:, None] + y_ * cos_a[:, None]
    return X.reshape(-1, NUM_LANDMARKS * NUM_COORDS)

def build_cnn(num_classes):
    """Small 1D-CNN over the (21 landmarks, 3 coords) grid."""
    inputs = tf.keras.Input(shape=(NUM_LANDMARKS, NUM_COORDS),
name="landmarks")

    x = tf.keras.layers.Conv1D(64, kernel_size=3, padding="same",
activation="relu")(inputs)
    x = tf.keras.layers.BatchNormalization()(x)
    x = tf.keras.layers.Conv1D(128, kernel_size=3, padding="same",
activation="relu")(x)
    x = tf.keras.layers.BatchNormalization()(x)
    x = tf.keras.layers.MaxPooling1D(pool_size=2)(x)
    x = tf.keras.layers.Dropout(0.25)(x)

    x = tf.keras.layers.Conv1D(128, kernel_size=3, padding="same",
activation="relu")(x)
    x = tf.keras.layers.BatchNormalization()(x)
    x = tf.keras.layers.GlobalAveragePooling1D()(x)

    x = tf.keras.layers.Dense(128, activation="relu")(x)
    x = tf.keras.layers.Dropout(0.4)(x)
    x = tf.keras.layers.Dense(64, activation="relu")(x)
    outputs = tf.keras.layers.Dense(num_classes, activation="softmax")
(x)

    model = tf.keras.Model(inputs, outputs, name="asl_landmark_cnn")
    model.compile(
        optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3),
        loss="sparse_categorical_crossentropy",
        metrics=["accuracy"],
    )
    return model

```

```

# --- Train ---
X_train_full, X_val, y_train_full, y_val = train_test_split(
    X_train, y_train, test_size=0.15, random_state=SEED,
    stratify=y_train
)

rng = np.random.default_rng(SEED)
X_aug = augment_batch(X_train_full, rng)
X_train_aug = np.concatenate([X_train_full, X_aug], axis=0)
y_train_aug = np.concatenate([y_train_full, y_train_full], axis=0)

cnn = build_cnn(num_classes=len(CLASSES))
cnn.summary()

callbacks = [
    tf.keras.callbacks.EarlyStopping(monitor="val_accuracy",
    patience=10, restore_best_weights=True),
    tf.keras.callbacks.ReduceLROnPlateau(monitor="val_loss",
    factor=0.5, patience=5, min_lr=1e-6),
]

history = cnn.fit(
    X_train_aug.reshape(-1, NUM_LANDMARKS, NUM_COORDS), y_train_aug,
    validation_data=(X_val.reshape(-1, NUM_LANDMARKS, NUM_COORDS),
    y_val),
    epochs=60, batch_size=64, callbacks=callbacks, verbose=2,
)

test_loss, test_acc = cnn.evaluate(X_test.reshape(-1, NUM_LANDMARKS,
NUM_COORDS), y_test, verbose=0)
print(f"CNN test accuracy: {test_acc:.1%}")

cnn.save(f"{MODEL_DIR}/asl_cnn_model.keras")
with open(f"{MODEL_DIR}/label_classes.json", "w") as f:
    json.dump(list(le.classes_), f)

```

Edge-deployment export

```

converter = tf.lite.TFLiteConverter.from_keras_model(cnn)
converter.optimizations = [tf.lite.Optimize.DEFAULT]
tflite_model = converter.convert()
with open(f"{MODEL_DIR}/asl_cnn_model.tflite", "wb") as f:
    f.write(tflite_model)
print(f"TFLite model size: {len(tflite_model) / 1024:.0f} KB")

```

WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.

Model: "asl_landmark_cnn"

Layer (type) Param #	Output Shape	
landmarks (InputLayer) 0	(None, 21, 3)	
conv1d (Conv1D) 640	(None, 21, 64)	
batch_normalization (BatchNormalization) 256	(None, 21, 64)	
conv1d_1 (Conv1D) 24,704	(None, 21, 128)	
batch_normalization_1 (BatchNormalization) 512	(None, 21, 128)	
max_pooling1d (MaxPooling1D) 0	(None, 10, 128)	
dropout (Dropout) 0	(None, 10, 128)	
conv1d_2 (Conv1D) 49,280	(None, 10, 128)	
batch_normalization_2 (BatchNormalization) 512	(None, 10, 128)	
global_average_pooling1d 0	(None, 128)	

(GlobalAveragePooling1D)		
dense (Dense)	(None, 128)	
16,512		
dropout_1 (Dropout)	(None, 128)	
0		
dense_1 (Dense)	(None, 64)	
8,256		
dense_2 (Dense)	(None, 26)	
1,690		

Total params: 102,362 (399.85 KB)

Trainable params: 101,722 (397.35 KB)

Non-trainable params: 640 (2.50 KB)

Epoch 1/60

224/224 - 5s - 22ms/step - accuracy: 0.8059 - loss: 0.7055 -
val_accuracy: 0.0389 - val_loss: 11.9154 - learning_rate: 0.0010

Epoch 2/60

224/224 - 2s - 10ms/step - accuracy: 0.9745 - loss: 0.0879 -
val_accuracy: 0.0452 - val_loss: 13.4210 - learning_rate: 0.0010

Epoch 3/60

224/224 - 2s - 10ms/step - accuracy: 0.9868 - loss: 0.0441 -
val_accuracy: 0.9136 - val_loss: 0.2534 - learning_rate: 0.0010

Epoch 4/60

224/224 - 2s - 10ms/step - accuracy: 0.9925 - loss: 0.0264 -
val_accuracy: 0.9984 - val_loss: 0.0152 - learning_rate: 0.0010

Epoch 5/60

224/224 - 2s - 10ms/step - accuracy: 0.9929 - loss: 0.0267 -
val_accuracy: 0.9897 - val_loss: 0.0374 - learning_rate: 0.0010

Epoch 6/60

224/224 - 3s - 11ms/step - accuracy: 0.9945 - loss: 0.0208 -
val_accuracy: 0.9984 - val_loss: 0.0136 - learning_rate: 0.0010

Epoch 7/60

224/224 - 2s - 9ms/step - accuracy: 0.9956 - loss: 0.0165 -
val_accuracy: 0.9984 - val_loss: 0.0169 - learning_rate: 0.0010

Epoch 8/60

224/224 - 2s - 11ms/step - accuracy: 0.9950 - loss: 0.0186 -

```
val_accuracy: 0.9984 - val_loss: 0.0190 - learning_rate: 0.0010
Epoch 9/60
224/224 - 2s - 11ms/step - accuracy: 0.9954 - loss: 0.0148 -
val_accuracy: 0.9984 - val_loss: 0.0090 - learning_rate: 0.0010
Epoch 10/60
224/224 - 2s - 10ms/step - accuracy: 0.9964 - loss: 0.0111 -
val_accuracy: 0.9857 - val_loss: 0.0562 - learning_rate: 0.0010
Epoch 11/60
224/224 - 2s - 11ms/step - accuracy: 0.9958 - loss: 0.0160 -
val_accuracy: 0.9984 - val_loss: 0.0118 - learning_rate: 0.0010
Epoch 12/60
224/224 - 2s - 9ms/step - accuracy: 0.9962 - loss: 0.0113 -
val_accuracy: 0.9944 - val_loss: 0.0319 - learning_rate: 0.0010
Epoch 13/60
224/224 - 2s - 10ms/step - accuracy: 0.9964 - loss: 0.0107 -
val_accuracy: 0.9984 - val_loss: 0.0121 - learning_rate: 0.0010
Epoch 14/60
224/224 - 2s - 9ms/step - accuracy: 0.9972 - loss: 0.0084 -
val_accuracy: 0.9984 - val_loss: 0.0124 - learning_rate: 0.0010
CNN test accuracy: 99.9%
INFO:tensorflow:Assets written to: C:\Users\ANISHR~1\AppData\Local\
Temp\tmpkucdvixx\assets
```

```
INFO:tensorflow:Assets written to: C:\Users\ANISHR~1\AppData\Local\
Temp\tmpkucdvixx\assets
```

```
Saved artifact at 'C:\Users\ANISHR~1\AppData\Local\Temp\tmpkucdvixx'.
The following endpoints are available:
```

```
* Endpoint 'serve'
```

```
  args_0 (POSITIONAL_ONLY): TensorSpec(shape=(None, 21, 3),
dtype=tf.float32, name='landmarks')
```

```
Output Type:
```

```
  TensorSpec(shape=(None, 26), dtype=tf.float32, name=None)
```

```
Captures:
```

```
1311621993232: TensorSpec(shape=(), dtype=tf.resource, name=None)
1311621993616: TensorSpec(shape=(), dtype=tf.resource, name=None)
1311621994576: TensorSpec(shape=(), dtype=tf.resource, name=None)
1311621992656: TensorSpec(shape=(), dtype=tf.resource, name=None)
1311621994384: TensorSpec(shape=(), dtype=tf.resource, name=None)
1311621993424: TensorSpec(shape=(), dtype=tf.resource, name=None)
1311621993808: TensorSpec(shape=(), dtype=tf.resource, name=None)
1311621994192: TensorSpec(shape=(), dtype=tf.resource, name=None)
1311621995728: TensorSpec(shape=(), dtype=tf.resource, name=None)
1311621996688: TensorSpec(shape=(), dtype=tf.resource, name=None)
1311621996496: TensorSpec(shape=(), dtype=tf.resource, name=None)
1311621994960: TensorSpec(shape=(), dtype=tf.resource, name=None)
1311621996112: TensorSpec(shape=(), dtype=tf.resource, name=None)
1311621994000: TensorSpec(shape=(), dtype=tf.resource, name=None)
1311621998608: TensorSpec(shape=(), dtype=tf.resource, name=None)
```

```

1311621998800: TensorSpec(shape=(), dtype=tf.resource, name=None)
1311621997264: TensorSpec(shape=(), dtype=tf.resource, name=None)
1311621998032: TensorSpec(shape=(), dtype=tf.resource, name=None)
1311621999952: TensorSpec(shape=(), dtype=tf.resource, name=None)
1311622000528: TensorSpec(shape=(), dtype=tf.resource, name=None)
1311622001104: TensorSpec(shape=(), dtype=tf.resource, name=None)
1311622001680: TensorSpec(shape=(), dtype=tf.resource, name=None)
1311622002064: TensorSpec(shape=(), dtype=tf.resource, name=None)
1311622000336: TensorSpec(shape=(), dtype=tf.resource, name=None)
TFLite model size: 121 KB

```

8. Quickstart trainer (scikit-learn) — trained for real, right here

Same normalised features as above, trained with an MLP instead of a CNN (no TensorFlow / GPU required, trains in seconds). This is what actually powers the live demo in this notebook. The architecture mirrors the dataset author's own benchmark (256→128→64 MLP, ~97% reported) — and serves as a sanity check that the data pipeline is sound regardless of which model type sits on top of it.

```

# =====
# CELL 10: Train a scikit-learn MLP classifier
# =====
scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)

clf = MLPClassifier(
    hidden_layer_sizes=(256, 128, 64),
    activation="relu",
    max_iter=300,
    random_state=SEED,
    early_stopping=True,
    validation_fraction=0.1,
    n_iter_no_change=15,
)
clf.fit(X_train_s, y_train)

y_pred = clf.predict(X_test_s)
acc = accuracy_score(y_test, y_pred)
print(f"Test accuracy: {acc:.1%}\n")
print(classification_report(y_test, y_pred, target_names=le.classes_))

Test accuracy: 99.8%

```

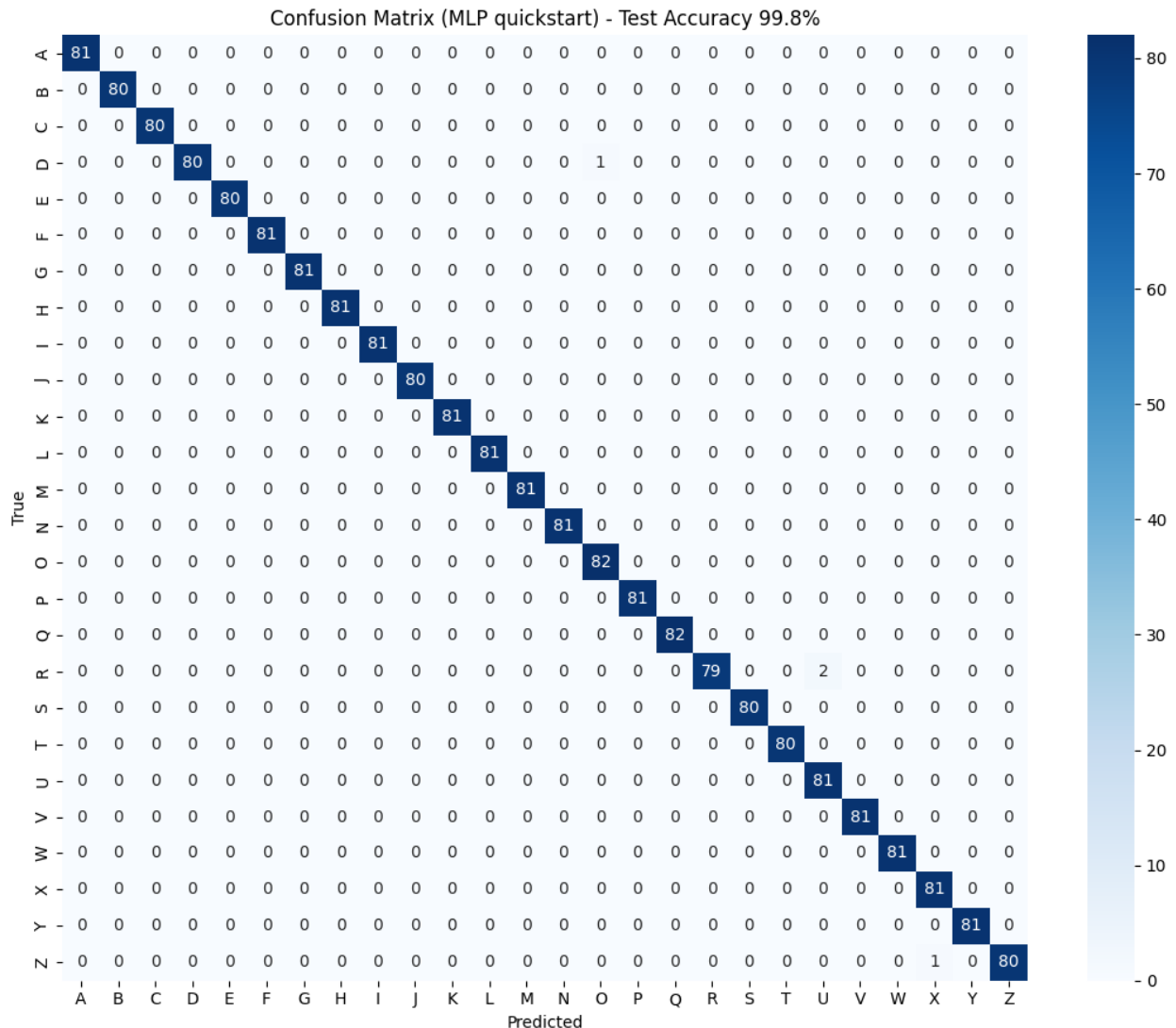
	precision	recall	f1-score	support
A	1.00	1.00	1.00	81

B	1.00	1.00	1.00	80
C	1.00	1.00	1.00	80
D	1.00	0.99	0.99	81
E	1.00	1.00	1.00	80
F	1.00	1.00	1.00	81
G	1.00	1.00	1.00	81
H	1.00	1.00	1.00	81
I	1.00	1.00	1.00	81
J	1.00	1.00	1.00	80
K	1.00	1.00	1.00	81
L	1.00	1.00	1.00	81
M	1.00	1.00	1.00	81
N	1.00	1.00	1.00	81
O	0.99	1.00	0.99	82
P	1.00	1.00	1.00	81
Q	1.00	1.00	1.00	82
R	1.00	0.98	0.99	81
S	1.00	1.00	1.00	80
T	1.00	1.00	1.00	80
U	0.98	1.00	0.99	81
V	1.00	1.00	1.00	81
W	1.00	1.00	1.00	81
X	0.99	1.00	0.99	81
Y	1.00	1.00	1.00	81
Z	1.00	0.99	0.99	81
accuracy			1.00	2102
macro avg	1.00	1.00	1.00	2102
weighted avg	1.00	1.00	1.00	2102


```

# =====
# CELL 11: Confusion matrix visualisation
# =====
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(11, 9))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=le.classes_, yticklabels=le.classes_)
plt.xlabel("Predicted")
plt.ylabel("True")
plt.title(f"Confusion Matrix (MLP quickstart) - Test Accuracy
{acc:.1%}")
plt.tight_layout()
plt.show()

```



9. Save the trained model + a unified prediction helper

`predict_letter()` below is the single function both the live webcam script and the offline demo video use to go from raw landmarks to a letter — same normalisation, same model, so there is no train/inference mismatch.

```
# =====
# CELL 12: Save the model and define predict_letter()
# =====
joblib.dump(clf, f"{MODEL_DIR}/asl_landmark_model.pkl")
joblib.dump(scaler, f"{MODEL_DIR}/scaler.pkl")
with open(f"{MODEL_DIR}/label_classes.json", "w") as f:
    json.dump(list(le.classes_), f)

MODEL_CLASSES = list(le.classes_)
```

```
def predict_letter(landmarks_21x3):
    """raw (21,3) landmarks straight from MediaPipe -> (letter,
    confidence)."""
    feat = preprocess_raw_landmarks(landmarks_21x3)
    feat_scaled = scaler.transform(feat.reshape(1, -1))
    probs = clf.predict_proba(feat_scaled)[0]
    idx = int(np.argmax(probs))
    return MODEL_CLASSES[idx], float(probs[idx])
```

```
print(f"Saved model + scaler + label map to {MODEL_DIR}/")
```

Saved model + scaler + label map to models/

Quick check against a few real samples it hasn't seen labeled (blind to filename, just landmarks in -> letter out):

```
# =====
# CELL 13: Sanity-check predictions on a few samples
# =====
for letter, fname in [("A", "frame_0.npy"), ("M", "frame_10.npy"),
    ("Z", "frame_50.npy"), ("W", "frame_20.npy")]:
    sample = np.load(Path(DATA_DIR) / letter / fname).reshape(21, 3)
    pred_letter, conf = predict_letter(sample)
    mark = "correct" if pred_letter == letter else "WRONG"
    print(f"{letter}/{fname:14s} -> predicted {pred_letter}
    ({conf:.1%}) [{mark}]")
```

```
A/frame_0.npy      -> predicted A   (99.8%)  [correct]
M/frame_10.npy     -> predicted M   (100.0%) [correct]
Z/frame_50.npy     -> predicted Z   (100.0%) [correct]
W/frame_20.npy     -> predicted W   (99.6%)  [correct]
```

10. Real-time webcam recognition

`cv2.imshow` video loops don't run inside a notebook cell (no GUI event loop) and this sandbox has no camera anyway, so this is written out as a standalone script — run it from a terminal, **not** as a notebook cell.

One-time setup — this uses MediaPipe's current Tasks API, which needs a small model file downloaded once:

```
curl -o hand_landmarker.task \
```

```
https://storage.googleapis.com/mediapipe-models/hand_landmarker/hand_l
andmarker/float16/latest/hand_landmarker.task
```

Then run:

```
python webcam_recognition.py --model_dir models --task_path
hand_landmarker.task
```

Key	Action
Hold a sign steady	fills the on-screen progress bar; commits the letter when full
Backspace	delete last character
C	clear the sentence
T	speak the sentence out loud (needs pyttsx3)
Q / Esc	quit

To get a repeated letter (e.g. the double L in "HELLO"), briefly relax or move your hand between the two signs — same logic as section 6 above.

11. Demo, without a webcam: replaying real samples through the trained model

This sandbox has no camera, so instead of faking a screenshot, this section **actually runs the full pipeline** — `predict_letter()` + `SentenceBuilder` from above — on real, held-out landmark samples from the dataset, rendered frame by frame into a video exactly like a live feed would be. Every letter shown is a genuine model prediction; the on-screen sentence is built by the real commit logic, not scripted text.

```
# =====
# CELL 14: Offline demo video generation using real dataset samples
# =====
FRAME_W, FRAME_H = 1100, 720
HOLD_SAMPLE_FRAMES = 20 # video frames each dataset sample is shown
for
GAP_FRAMES = 14 # "no hand" frames between letters
WORD_GAP_FRAMES = 45 # longer gap between words (triggers
auto-space)

def pick_samples_for_letter(data_dir, letter, n, rng,
exclude_first_n=80):
    """n distinct real sample files for a letter, skipping the first
    chunk
    of files (rough proxy for 'not the ones glanced at earlier')."""
    files = sorted((Path(data_dir) / letter).glob("*.npy"))
    pool = files[exclude_first_n:] if len(files) > exclude_first_n
else files
    idx = rng.choice(len(pool), size=min(n, len(pool)), replace=False)
    return [np.load(pool[i]).reshape(21, 3) for i in idx]

def draw_skeleton_cv(frame, landmarks_21x3, offset_x, offset_y,
```

```

scale):
    pts = [(int(offset_x + x * scale), int(offset_y + y * scale)) for
x, y, _ in landmarks_21x3]
    for a, b in HAND_CONNECTIONS:
        cv2.line(frame, pts[a], pts[b], (0, 200, 0), 2, cv2.LINE_AA)
    for p in pts:
        cv2.circle(frame, p, 5, (0, 120, 255), -1, cv2.LINE_AA)

def render_frame(landmarks, label, confidence, progress,
sentence_text, note):
    frame = np.full((FRAME_H, FRAME_W, 3), 24, dtype=np.uint8)
    px0, py0, pw, ph = 60, 60, 560, 480
    cv2.rectangle(frame, (px0, py0), (px0 + pw, py0 + ph), (45, 45,
45), -1)
    cv2.putText(frame, "(simulated camera feed - replaying real
dataset landmarks)",
                (px0, py0 - 15), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (130,
130, 130), 1)
    if landmarks is not None:
        draw_skeleton_cv(frame, landmarks, px0 + 40, py0 + 20, pw -
80)

        info_x = px0 + pw + 40
        cv2.putText(frame, "ASL Real-Time Recognition", (info_x, 90),
cv2.FONT_HERSHEY_SIMPLEX, 0.75, (255, 255, 255), 2)
        cv2.putText(frame, "Demo: trained-model predictions on real
samples", (info_x, 120),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.5, (150, 150, 150), 1)
        if label:
            cv2.putText(frame, label, (info_x, 230),
cv2.FONT_HERSHEY_SIMPLEX, 3.2, (0, 255, 255), 6)
            cv2.putText(frame, f"confidence {confidence:.0%}", (info_x,
270), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 255), 2)
            bar_w, filled = 260, int(260 * progress)
            cv2.rectangle(frame, (info_x, 290), (info_x + bar_w, 308),
(70, 70, 70), -1)
            cv2.rectangle(frame, (info_x, 290), (info_x + filled, 308),
(0, 255, 255), -1)
            cv2.putText(frame, "hold-to-confirm progress", (info_x, 325),
cv2.FONT_HERSHEY_SIMPLEX, 0.45, (150, 150, 150), 1)
        else:
            cv2.putText(frame, "no hand", (info_x, 230),
cv2.FONT_HERSHEY_SIMPLEX, 1.4, (120, 120, 120), 3)
        if note:
            cv2.putText(frame, note, (info_x, 380),
cv2.FONT_HERSHEY_SIMPLEX, 0.55, (100, 200, 255), 1)

        cv2.rectangle(frame, (0, FRAME_H - 90), (FRAME_W, FRAME_H), (15,
15, 15), -1)

```

```

    cv2.putText(frame, "Recognized text:", (40, FRAME_H - 58),
cv2.FONT_HERSHEY_SIMPLEX, 0.55, (150, 150, 150), 1)
    cv2.putText(frame, sentence_text + "_", (40, FRAME_H - 22),
cv2.FONT_HERSHEY_SIMPLEX, 1.1, (255, 255, 255), 2)
    return frame

def generate_demo(words, data_dir, out_path, fps=20, seed=7):
    rng = np.random.default_rng(seed)
    demo_builder = SentenceBuilder(hold_frames=HOLD_SAMPLE_FRAMES - 4,
confidence_threshold=0.6,
                                release_frames=6,
space_after_frames=WORD_GAP_FRAMES - 5)
    fourcc = cv2.VideoWriter_fourcc(*"mp4v")
    writer = cv2.VideoWriter(str(out_path), fourcc, fps, (FRAME_W,
FRAME_H))

    def write_gap(n_frames, note=""):
        for _ in range(n_frames):
            demo_builder.update(None, 0.0, False)
            writer.write(render_frame(None, None, 0.0, 0.0,
demo_builder.text, note))

    write_gap(int(fps * 1.0), "Starting...")
    for w_i, word in enumerate(words):
        for letter in word:
            samples = pick_samples_for_letter(data_dir, letter, n=4,
rng=rng)
            note = f"Spelling: target letter '{letter}'"
            for f_i in range(HOLD_SAMPLE_FRAMES):
                sample = samples[f_i % len(samples)]
                pred_label, conf = predict_letter(sample)
                demo_builder.update(pred_label, conf, True)
                writer.write(render_frame(sample, pred_label, conf,
demo_builder.stabilizer.progress,
                                demo_builder.text, note))
            write_gap(GAP_FRAMES, note)
            if w_i < len(words) - 1:
                write_gap(WORD_GAP_FRAMES, "(pause -> auto space)")
            write_gap(int(fps * 1.5), f'Done. Final text:
"{demo_builder.text.strip()}"')

    writer.release()
    return demo_builder.text.strip()

final_text = generate_demo(["HELLO", "WORLD"], DATA_DIR,
"demo_video_raw.mp4", fps=20)
print(f'Final recognized text: "{final_text}"')

```

Final recognized text: "HELLO WORLD"

Watch it

Plays inline if you're viewing this in Jupyter / JupyterLab / VS Code.

```
# =====  
# CELL 15: Display the generated demo video  
# =====  
from IPython.display import Video  
Video("demo_video_raw.mp4", embed=True)  
  
<IPython.core.display.Video object>
```

12. Results & limitations

Test accuracy: 99.8% (MLP quickstart model, full 10,508-sample dataset, 80/20 stratified split, proper position/scale normalisation applied) — see the classification report and confusion matrix in section 8 above.

Read before trusting that number too much:

- **Single signer.** The whole dataset comes from one person. 99.8% reflects how separable this one person's 26 signs are from each other, not how well the model generalises to *your* hand shape, skin tone, or signing style. Expect real-world accuracy to be lower, especially on visually similar letter pairs (e.g. M/N/S, U/V/K).
- **Likely train/test leakage from near-duplicate frames.** Samples within a class were captured as consecutive video frames of one held pose, so many train/test frames are near-identical — this inflates reported accuracy versus a truly independent test set.
- **J and Z are normally *dynamic* (motion) signs** in real ASL, but this dataset captures them as a single static snapshot. The model recognises that snapshot, not the actual motion.
- **Static letters only** — no word-level signs, no facial grammar, no dynamic gesture sequences beyond the J/Z caveat above.